

Complex Python

Lesson #2

By: Alan Alappatt

Table Of Contents

Downloading Pycharm 01

While Loops 02-03

For Loops 04

Range Function 05

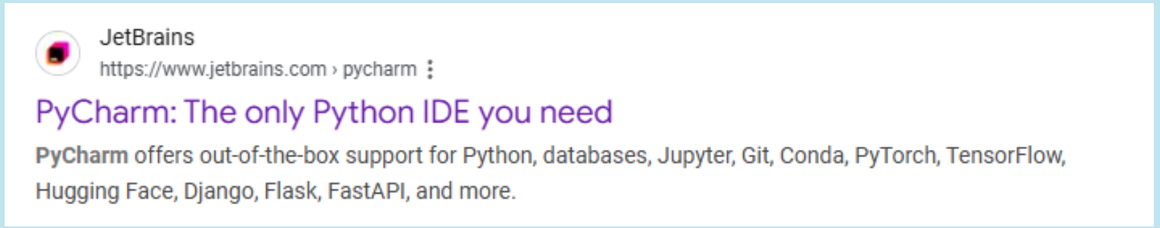
Loop Control 06

Lists 07

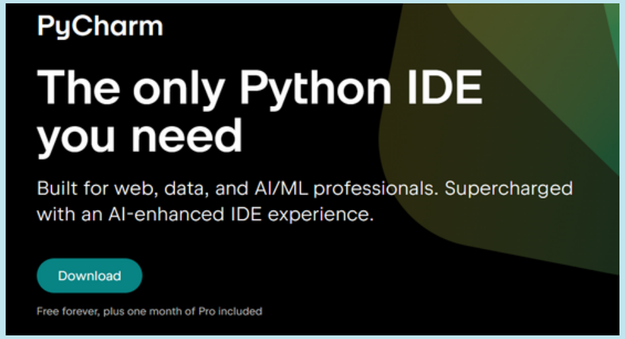
Downlaoding Pycharm

If you have not already download Pycharm, please do it now.

Search up Pycharm



Download this



Go through the download setup

While Loops

```
counter = 0
while counter < 10:
    print(counter)
    counter += 2
```

```
0
2
4
6
8
```

While loops will continuously execute a block of code as long as as the condition is True. This is best used in situations where you don’t know beforehand how much times the loops needs to run, like when you need to check for the user to input the password. This could be used to monitor a phone’s battery. While the battery life below 20%, put the phone in low power mode.

Try making your own now!

While Loop Question

```
counter = 3
while counter < 15:
    print(counter)
    counter += 5
```

```
3
8
13
```

```
counter = 0
while counter < 7:
    print(counter + 2)
    counter += 3
```

```
2
5
8
```

For Loops

For Loops are best used for iterating over a list or series of items. This is usaully used when you know how many times you need to repeat the code.

Use # to make a comment

Putting f tells Pycharm to replace the variable name with the variable's value.
Also, doing print("Iteration: " + str(i)) will work.

Range() will start at the 1st value and end before the last value.

```
#shows number of iteration
for i in range(1, 6):
        print(f"Iteration: {i}")
```

```
Iteration: 1
Iteration: 2
Iteration: 3
Iteration: 4
Iteration: 5
```

Range Function

Range() is very important in for loop iteration. There are three types of range():

range(number) will start from 0 and stop right before the number.

range(num1, num2) will start at num1 and stop before num2.

range(num1, num2, num3) will start at num1 and stop before num 2, but will increment by num3
Ex. range(1, 5, 2)
1 3

Loop Control

The 2 things you should know are the break and continue keywords. Break will stop the code and continue will skip that specific iteration. Both are best used in an if statement.

```
for i in range(1, 5):
    if i == 3:
        print("Loop Break")
        break
    print(i)
```

```
1
2
Loop Break
```

```
for i in range(0, 10, 3):
    if i == 3:
        continue  # Skips 3
    print(i)
```

```
0
6
9
```

Activity Time!

Review Questions

```python
for i in range(3,12):
    if(i == 9):
        break
    elif(i == 7 or i == 5):
        continue
    print(i * 2)
```

```
6
8
12
16
```

```python
count = 5
while 5 >= count >= 0:
    count -= 1
    if count == 4 or count == 1:
        continue
    else:
        print(count)
```

```
3
2
0
-1
```

Thank You!

Any Questions?